



AI Assisted Development Program

Day 2

AI Adoption, Plan Mode & Spec-Driven Development

Agenda For Today



AI-Assisted Coding & Adoption

What research says about AI and skill-building



MCP & Hooks

Extending Claude Code — section in progress



Claude Code Plan Mode

Separate thinking from doing



Spec-Driven Development

Make intent the source of truth



PART 1

AI-Assisted Coding & AI Adoption

What a controlled study reveals about speed, skills — and how you should use AI.

THE RESEARCH

How AI Assistance Impacts Coding Skills

A new Anthropic study puts the speed-vs-learning question to a controlled test.



“How AI Impacts Skill Formation”

Shen, J. H. & Tamkin, A. (2026)

Anthropic · Jan 29, 2026 · arXiv:2601.20245

arxiv.org/abs/2601.20245



What it tests

- Does AI speed come at the cost of skill formation?
- 52 engineers in a randomized controlled trial
- Mastery measured: debugging · reading · writing · concepts

Does AI Help or Hurt Skill Development?



AI makes people faster

Earlier Anthropic work found speed-ups of up to 80% on some tasks.



Speed has a hidden cost

Leaning on AI, people tend to disengage and offload their thinking.



The core tension

As AI writes more code, humans still must catch errors and provide oversight.



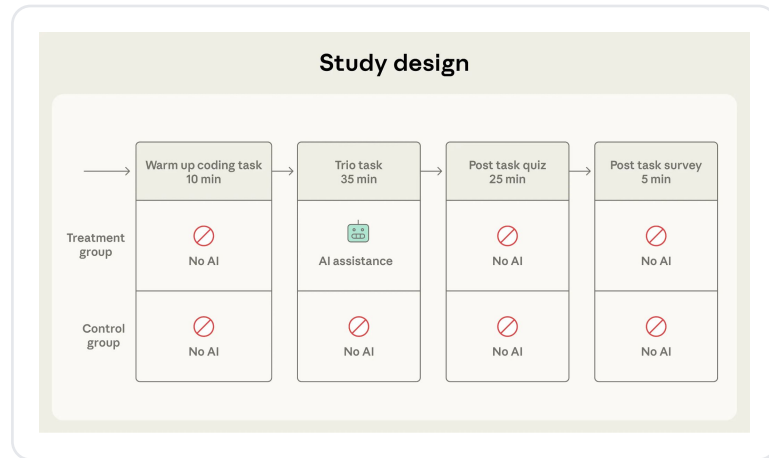
So the study asks

Is AI a shortcut to efficiency AND learning — or do gains undermine skill-building?

THE METHOD

How the Study Worked

- **Randomized controlled trial:** 52 mostly-junior engineers — regular Python users, all new to the Trio async library.
- **Three parts:** a warm-up, a main task (build two features with Trio), then a quiz on the concepts just used.
- **Two groups:** one with an AI assistant that could generate correct code on request; one coding by hand.
- **Four mastery types:** debugging, code reading, code writing, and concepts — weighted toward the skills needed to oversee AI-written code.



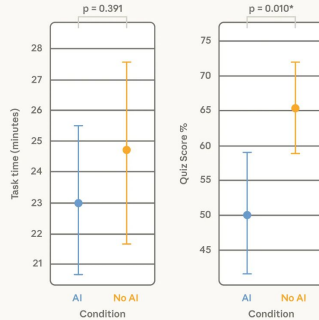
*Study design — Shen & Tamkin (2026), *arXiv:2601.20245*

THE KEY FINDING

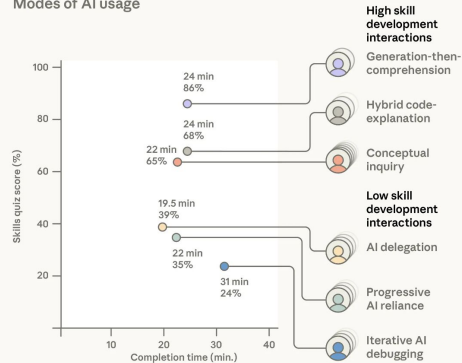
A Real Trade-off

How AI assistance impacts coding speed and skill formation

AI assistance: treatment effect on coding speed and knowledge score



Modes of AI usage



17% lower

AI-group quiz score — about two letter grades below the hand-coding group.

~50% vs ~67%

Average scores, AI vs. hand-coding (Cohen's $d = 0.738$, $p = 0.01$).

Debugging

The biggest gap — exactly the skill needed to spot when AI code is wrong.

Only ~2 min faster

AI's speed gain on this unfamiliar task — not statistically significant.

*Quiz results — AI group vs. hand-coding group

WHAT ACTUALLY MATTERS

It's HOW You Use AI

Using AI didn't guarantee worse learning — the interaction style determined the outcome.

LOW SCORERS · <40%

- Fully delegating code generation to AI
- Sliding into delegation over time
- Using AI only to debug or fix errors

HIGH SCORERS · 65%+

- Generate code, then ask follow-up questions
- Request code together with explanations
- Ask conceptual questions while coding solo

Takeaway Cognitive effort — even getting stuck — builds mastery. Use learning-oriented modes (Explanatory mode, Study Mode) and deploy AI intentionally so people keep learning while they work.



QUICK PULSE

How Are You Using AI?

Drop it in the chat — anyone want to share their workflow before we move on?



REMEMBER THIS

Direction. Context. Oversight.

AI is a powerful assistant that needs all three — never autonomous judgment.

An AI Adoption Journey in Stages

Adopting any real tool means pushing through three phases: **inefficiency** → **adequacy** → **workflow-altering discovery**.

1

Drop the chatbot

Copy-pasting to chat is inefficient — use an agent that reads files and runs programs.

2

Reproduce your own work

Redo manual commits with the agent to learn its limits — and when not to use it.

3

End-of-day agents

Kick off research or triage in the last 30 min for a warm start tomorrow.

4

Outsource the slam dunks

Delegate what AI reliably nails; keep your deep work. You decide when to switch.

5

Engineer the harness

Turn every mistake into a rule (AGENTS.md) or a real verification tool.

6

Keep an agent running

Aspirational: a useful background task — only when genuinely valuable.

A Disciplined AI-Augmented Workflow

1

Plan before code

Brainstorm a spec.md with the AI, then a step-by-step plan — a “waterfall in 15 minutes.”

2

Break work into small chunks

One function, bug, or feature at a time — monolithic asks produce a jumbled mess.

3

Pack in context

Show the relevant code, docs, and constraints — the AI is only as good as what you give it.

4

Pick the right model

Each has a “personality” — swap models when one gets stuck; use the best tier available.

Human in the Loop

1

Verify, test, review everything

Treat AI output like a junior dev's — confident but error-prone. Never merge code you can't explain.

2

Commit often

Frequent commits are “save points” you roll back to when an AI change goes sideways.

3

Customize behavior

Rules files (CLAUDE.md), style guides, and inline examples keep output on-convention.

4

Automation as a force multiplier

Strong CI, tests, and linters let the AI self-correct in a tight feedback loop.

Big takeaway AI amplifies fundamentals — strong engineers get multiplied; weak foundations get amplified confusion. Keep coding without AI sometimes. The human stays the director of the show.



PART 2

MCP & Hooks

Connecting Claude Code to your tools — and automating your guardrails.

MCP: What It Is & Why It Exists

An open standard that connects Claude Code to external tools, services & data — turning a self-contained assistant into a versatile agent.



File Systems

Local dev environments & codebases.



Databases

Internal data & structured info.



Third-Party APIs

GitHub, Slack, Jira, Notion & more.



Browser & Web

Automated browsing & web actions.



Custom Tools

Proprietary internal systems.



Search Engines

Live retrieval across the web.

Like USB-C — before it, every device needed its own cable. MCP is one universal connector for AI agents and the tools they use.

HOW IT'S USED

MCP in Practice

Claude connects to an MCP server — a lightweight process exposing tools and data. It calls those tools like any native skill: request, receive, integrate, move on.



Tools

Actions Claude can execute in external systems — create a GitHub PR or query a database.



Resources

Data Claude can read and incorporate — pull a Notion doc or read a log file.



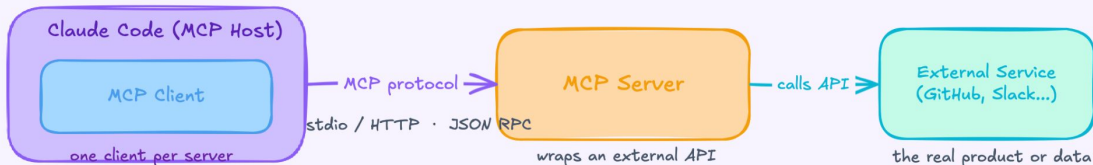
Prompts

Reusable templates and pre-built workflows for common tasks — consistency & efficiency.

How Claude Code Connects to the World

Model Context Protocol (MCP)

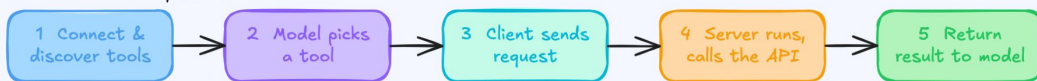
1. The MCP architecture



2. What an MCP server exposes



3. The call flow, end to end



Example servers in the MCP ecosystem: Github, Slack, Postgres, Sentry, Google Drive, Filesystem

Fixing a Bug with MCP

Claude Code is tasked with fixing a Jira bug and opening a Pull Request — orchestrating tools and MCP servers end to end.

1

MCP · Jira

Pulls ticket details into context.

2

Read File

Loads the relevant source file.

3

Edit File

Applies the code fix directly.

4

Bash

Runs tests to validate the fix.

5

MCP · GitHub

Opens a PR with a commit message.

6

MCP · Slack

Notifies the team about the PR.

LIVE DEMO

Live Demo: MCP

claude — demo session

In this demo, we will:

- Build an MCP server in TypeScript exposing tools, resources & prompts over a doc store.
- Define `read_doc_contents` and `edit_document` tools using the SDK and Zod schemas.
- Create an MCP client that lists & calls tools, reads resources, fetches prompts.
- Test everything end-to-end with the MCP Inspector.



Link for

Hands-on workshop

[CLICK HERE](#)

WHAT & WHY

Hooks: What They Are

User-defined shell commands Claude Code runs automatically at specific points — intercept, observe, and control behavior without altering the core agent.



PreToolUse

Before any tool call —
pre-execution checks or
modifications.



PostToolUse

After a tool completes —
logging, validation,
follow-up.



Notification

On a status update —
custom alerts or progress
tracking.



Stop

When a task finishes —
cleanup, reporting, next
steps.

The Core Idea Instead of prompting Claude to “always run the linter,” you wire it in. Claude edits → hook fires → linter runs. Guaranteed, every time — no tokens, no ambiguity.

Why Hooks Matter

1

What Hooks Can Do

- Run shell commands, scripts, or any executable
- Read tool call details: tool, inputs & outputs
- Block an action before it happens (PreToolUse)
- Trigger side effects after an action completes

2

Why This Matters Architecturally

- Operate outside Claude's context window
- Add zero tokens and create zero ambiguity
- Never depend on Claude interpreting instructions
- Enforcement is structural, not conversational

Hooks in the Claude Code Lifecycle

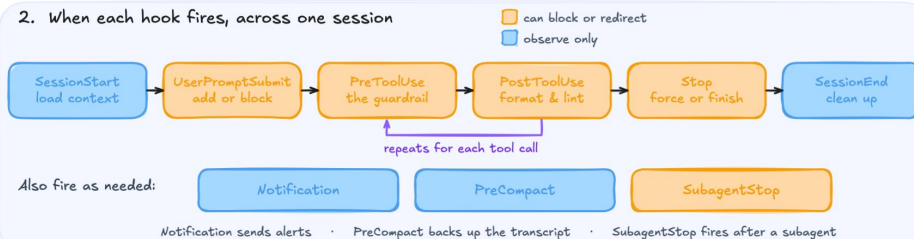
Run your own commands at key moments in the agent loop

1. What a hook is

A hook is a command Claude Code runs automatically when a lifecycle event fires.
It can observe, block, or extend the agent without changing the model.

Configured in `.claude/settings.json` or `/hooks` · handlers: command, http, prompt, agent

2. When each hook fires, across one session



3. How a hook controls the action



Core events shown; Claude Code defines more, such as `PermissionRequest` and `UserPromptExpansion`.

Live Demo: Hook

claude — demo session

In this demo, we will:

- Run hooks that always fire at key points — format, log, notify, or block.
- Block bad actions with a PreToolUse hook (exit 2 / “deny”) — e.g. reading .env.
- React after a tool runs with PostToolUse — auto-format, type-check, catch dupes.
- Configure in settings.json and commit project hooks so the whole team gets them.



Link for

Hands-on workshop

[CLICK HERE](#)



PART 3

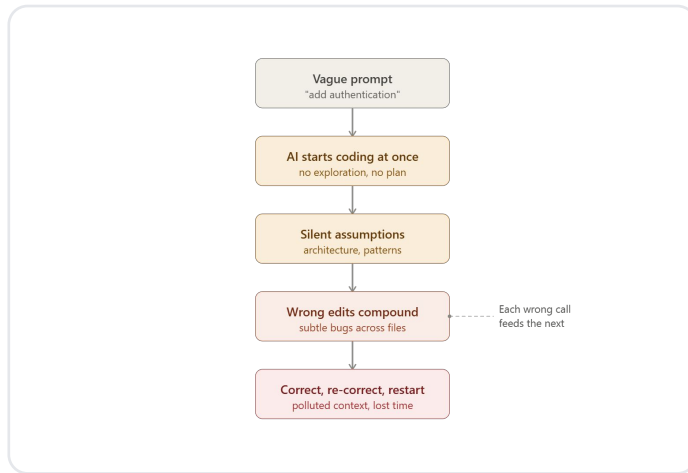
Claude Code Plan Mode

Separate thinking from doing — front-load the decisions, then execute fast.

THE PROBLEM

Why Plan Before You Build

- **Assistants are trained to be helpful** — they start writing code immediately, making silent assumptions about your architecture and patterns.
- **Assumptions compound**: $\sim 80\%$ right per decision $\times \sim 20$ decisions in a feature $\approx$ about a 1% chance everything is correct.
- **The familiar loop**: fast output $\rightarrow$ subtle bugs $\rightarrow$ correction after correction $\rightarrow$ polluted context $\rightarrow$ start over.
- **The fix isn't a cleverer prompt** — it's separating thinking from doing.



**How unguided assumptions compound*

DEFINITION

What Plan Mode Is

A **read-only operating mode**: Claude reads files, searches the codebase, and asks clarifying questions — but cannot change your project.

ALLOWED

- Read files · search (glob / grep)
- List directories · web search & fetch
- Spawn research sub-agents
- Ask multiple-choice clarifying questions

BLOCKED

- Writing or editing files
- Running shell commands
- Anything that changes your project

The output A reviewable plan — nothing is implemented until you approve it. Thinking is separated cleanly from execution.

The Three Modes — and How to Enter



Normal Mode

Asks permission before each edit.
Best for small, reviewable changes.



Auto-Accept Mode

Executes without asking. Best for
batch-running an approved plan.



Plan Mode

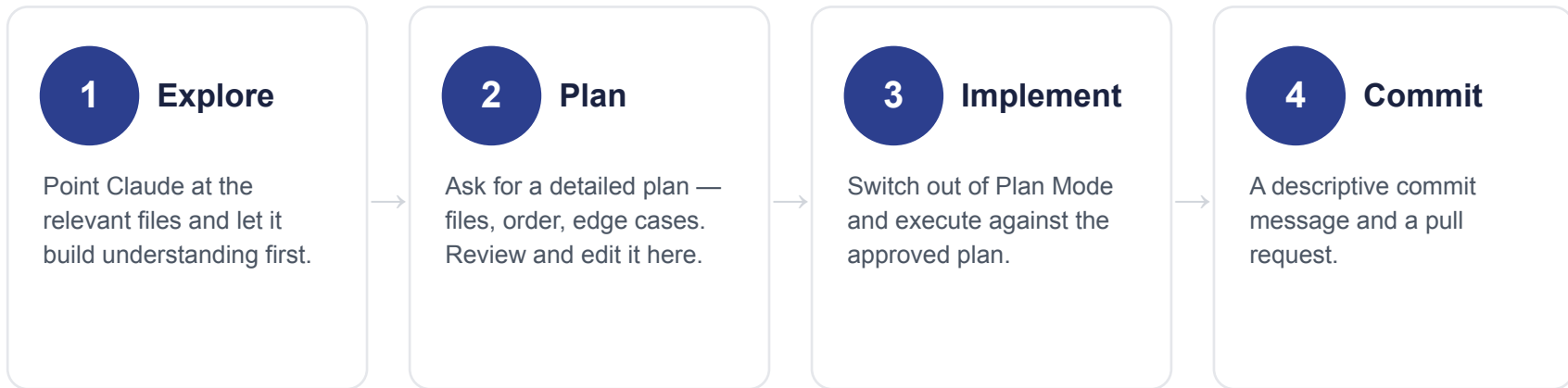
Read-only analysis. Best for
complex, multi-file, architectural, or
unfamiliar work.

Enter Plan Mode: Shift+Tab twice · /plan mid-session · launch with --permission-mode plan · or set as default (Alt+M on some Windows terminals).

THE CORE WORKFLOW

Explore → Plan → Implement → Commit

The first two phases run in Plan Mode — review happens before a single line changes.



Key insight: The first two phases are the cheapest in tokens and the most valuable in outcome — they prevent solving the wrong problem.

Review the Plan — and Know When to Skip

THE PLAN IS A CONTRACT

- Open it in your editor (Ctrl+G): annotate, delete steps, reorder — Claude picks up every change.
- Add missing constraints before approving.
- Store plans in your repo — they become decision records and ready-made PR descriptions.

WHEN TO SKIP

- If you can describe the exact change in one sentence — just do it.
- Plan when the task touches multiple files, involves an architectural decision, or the codebase is unfamiliar.

Rule of thumb: 1–2 files & obvious → skip. 3+ files or any ambiguity → plan first.

Getting the Most Out of Plan Mode

1

Pair it with a good **CLAUDE.md**

Plan quality tracks your documented conventions — Claude reads them while planning and follows them from the start.

2

Give Claude a way to verify

Tests, a build, or a screenshot comparison let it self-correct instead of relying on you to catch every mistake.

3

Interleave tests with implementation

Favor TDD-style plans over testing at the end — each passing test confirms you're on track.

4

Watch for drift during execution

Edits outside the current step? Drop back into Plan Mode and re-plan the rest.



THE POINT

Front-Load the Thinking

Plan Mode isn't about slowing down — it makes execution fast, predictable, and correct. It's the discipline of reviewing a spec before you build, enforced by the tool.

Live Demo: Claude Code Plan Mode

claude — plan mode demo

In this demo, we will:

- Enter Plan Mode and explore the codebase read-only.
- Generate a plan, then edit it — annotate, reorder, add constraints.
- Approve, execute with auto-accept, and commit the result.



Link for

Hands-on workshop

[CLICK HERE](#)

What best describes Plan Mode?

A

A mode that auto-executes edits without asking

B

A read-only mode: the agent can read, search, and ask — but not change the project

C

A faster model for simple fixes

D

A mode that commits code automatically after each step

What best describes Plan Mode?

A

A mode that auto-executes edits without asking

B

A read-only mode: the agent can read, search, and ask — but not change the project



C

A faster model for simple fixes

D

A mode that commits code automatically after each step

According to the “one-sentence rule,” when should you skip planning?

A When the task touches many files

B When you're unfamiliar with the codebase

C When you can describe the exact change in a single sentence

D When an architectural decision is involved

According to the “one-sentence rule,” when should you skip planning?

A When the task touches many files

B When you're unfamiliar with the codebase

C When you can describe the exact change in a single sentence ✓

D When an architectural decision is involved



PART 4

Spec-Driven Development

From vibe coding to executable intent — the spec becomes the source of truth.

From Vibe Coding to Spec-Driven

- **Vibe coding** — describe a goal, get code back — is great for quick prototypes, but unreliable for serious, mission-critical work or existing codebases.
- **The problem isn't the agent's ability**; it's our approach: we treat literal-minded pair programmers like search engines.
- **SDD flips the model**: you write the spec first, and the agent implements against it.

The Big Idea

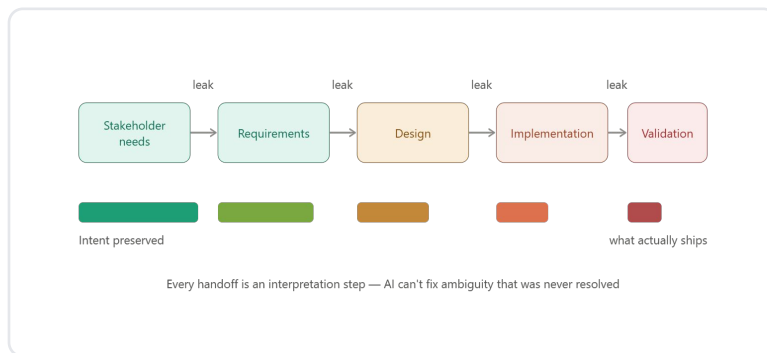


- From “code is the source of truth”...
- **...to “intent is the source of truth.”**
- AI is what makes a specification executable.

THE FAILURE MODE

Why Vibe Coding Breaks Down

- **“Add photo sharing to my app”** forces the model to guess thousands of unstated requirements — wrong guesses surface deep into implementation.
- **Requirements scattered across prompts** mean no durable source of truth → architectural drift, inconsistency, harder reviews, rework.
- **AI accelerates every handoff**, but it cannot fix ambiguity that was never resolved.

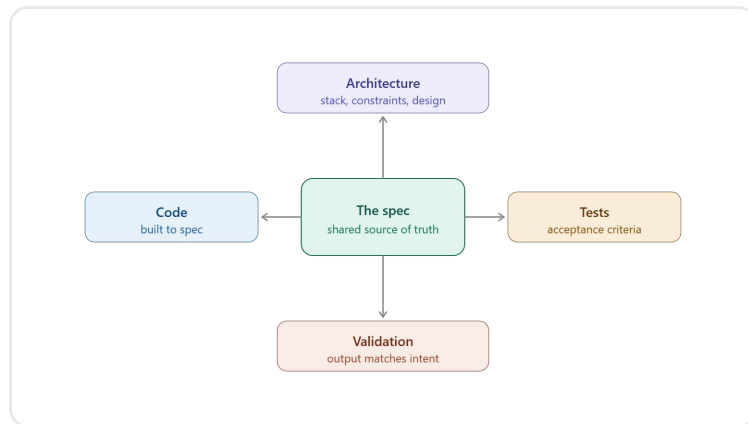


**Every handoff is an interpretation step — AI can't fix ambiguity that was never resolved*

DEFINITION

What Spec-Driven Development Is

- **Spec-first:** define the guardrails, requirements, constraints, acceptance criteria, and edge cases up front.
- **A living, executable artifact** — the spec is the shared source of truth for both humans and AI, not a static document.
- **Connective tissue** linking business intent to architecture, implementation, tests, and validation.
- **Align first**, then let AI accelerate execution from a clear spec — instead of prompting first and aligning later.



**The spec as the hub of the lifecycle*

The Spec-Driven Workflow

1

Constitution

Principles, standards, guardrails, tech stack
— persists across agent sessions.

2

Specify

Capture the what & why: user journeys,
success criteria. No tech stack yet.

3

Clarify

Resolve ambiguity, dependencies, and edge
cases before planning.

4

Plan

Now go technical: stack, architecture,
constraints, performance targets.

5

Tasks

Break the plan into small, independently
testable chunks.

6

Implement → Validate

The agent builds task by task; you verify
against the spec.

Key discipline: Each phase has one job — don't advance until the current one is validated. The core four everyone shares: Specify → Plan → Tasks → Implement.

WHY IT WORKS

Small Tasks — “What” vs. “How”

- **Concrete, reviewable tasks** the agent can implement and test in isolation — almost like TDD for the AI.
- **Focused reviews:** changes that solve specific problems — not thousand-line code dumps.
- **Stable “what,” flexible “how”:** to change course, update the spec and regenerate the plan — no expensive rewrite.
- **Organizational bonus:** security, compliance, and design-system rules live in the spec — where AI can actually use them.

VAGUE — AGENT GUESSES

“Build authentication”

CONCRETE — AGENT VERIFIES

“Create a registration endpoint that validates email format”

Where It Shines & What Determines Quality



Greenfield · 0→1

The agent builds what you intend
— not a generic pattern.



Feature work · N→N+1

Its most powerful use — the spec
forces clarity on how new code fits
the system.



Legacy modernization

Recapture lost intent in a fresh
spec; rebuild without inherited tech
debt.

Spec quality = output quality. Capture intent, constraints, and acceptance criteria. One reported result: onboarding a new module type dropped from 2–3 weeks to a few days. Right-size it — keep specs lightweight and living.

Stay the Human in the Loop

SDD is not “edit a spec and hope” — blindly handing everything to specs means losing control of your own codebase. Steer and verify at every checkpoint.



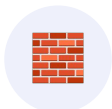
“Grill me” sessions

Have the AI interview you to force a shared design before any code is written.



Vertical slices + TDD

Thin end-to-end features and failing-tests-first stop the agent from coding blind.



Structure code for AI

Favor deep modules — you design the interfaces, delegate the internals.



Watch for doc rot

Retire old specs and PRDs so the agent doesn't build against stale requirements.



THE BOTTOM LINE

Intent Is the Limiting Factor

Speed alone doesn't guarantee better software. As work becomes AI-native, what matters is how clearly intent can be captured, shared, and validated — the spec is how you do that.

Live Demo: Spec-Driven Development

claude — sdd demo

In this demo, we will:

- Write a spec.md — requirements, constraints, acceptance criteria.
- Generate the plan and break it into small, testable tasks.
- Implement task by task and validate output against the spec.



Link for

Hands-on workshop

[CLICK HERE](#)

What core shift does spec-driven development represent?

A

From “intent is the source of truth” back to “code is the source of truth”

B

From writing tests first to writing tests last

C

From “code is the source of truth” to “intent is the source of truth”

D

From using agents to avoiding them entirely

What core shift does spec-driven development represent?

A

From “intent is the source of truth” back to “code is the source of truth”

B

From writing tests first to writing tests last

C

From “code is the source of truth” to “intent is the source of truth” ✓

D

From using agents to avoiding them entirely

Why does the spec-driven workflow break work into small, concrete tasks?

A

So they can be implemented and tested in isolation, keeping the agent on track

B

So the developer never has to review the output

C

Because agents can only read one file at a time

D

To make the spec longer and more detailed

Why does the spec-driven workflow break work into small, concrete tasks?

A

So they can be implemented and tested in isolation, keeping the agent on track



B

So the developer never has to review the output

C

Because agents can only read one file at a time

D

To make the spec longer and more detailed



Thank You!

That's a wrap on Day 2 — AI adoption, Plan Mode & spec-driven development



**AI Assisted Development
Program · Day 2**



Advanced Assignment (Optional)



Link for
Hands-on Assignment

[CLICK HERE](#)